Laboratório Funcional · Aula 12

Sergio Guilherme Reis Pedroso · RA 241399322 · Turma ESOFT6S-N-B
Gerado em 16/10/2026, 20:08:16 · tempo registrado no cronômetro: 37:06

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

agora(): É impura porque quebra a transparência referencial.
poe(lista, x): É impura porque gera um efeito colateral. Em vez de criar e devolver uma nova lista modificada, o método lista.append(x) altera diretamente a lista original.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

Linha 1: 1: A-1 A-2
Linha 4: 4: 14
Linha 7: 7: [7, 3, 9] [3, 7, 9]

Saídas (original e com compor(g, f))

```
original:
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]

compor(g, f):
1: A-1 A-2
```

```
2: True
3: 8
4: 18
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

A função proximo_id devolve valores diferentes porque ela é impura e depende de um estado externo modificável (a variável global contador). A cada execução, o estado interno da variável é alterado (efeito colateral).
Isso dificulta os testes automatizados porque a função perde a transparência referencial

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

compor(f, g)(4) = $f(g(4))$: Primeiro avalia-se $g(4) = 3 \times 4 = 12$.
Depois, aplica-se f ao resultado: $f(12) = 12 + 2 = 14$.

compor(g, f)(4) = $g(f(4))$: Primeiro avalia-se $f(4) = 4 + 2 = 6$.
Depois, aplica-se g ao resultado: $g(6) = 3 \times 6 = 18$.

Explicação: A composição de funções não é comutativa

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

Diferença: A função sorted(notas) cria e retorna uma nova lista contendo os elementos ordenados, mantendo a lista original notas intacta. Já o método notas.sort() altera a própria lista original diretamente (in-place).
Qual segue o estilo funcional: O sorted(notas) segue o estilo funcional pois os dados devem ser imutáveis e as funções devem ser puras

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

(B C)

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

```
original:
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
```

```
B

sem apostrofo:
22
14
25
8
negativo
Backtrace:
           5 (primitive-load "/box/main.scm")
In ice-9/eval.scm:
   191:35  4 (_ #f)
   191:35  3 (_ #f)
   196:27  2 (_ #f)
   223:20  1 (proc #<directory (guile-user) 143ceb4d1c80>)
In unknown file:
           0 (%resolve-variable (7 . A) #<directory (guile-user) 143…>)

ERROR: In procedure %resolve-variable:
Unbound variable: A
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo serve para indicar ao interpretador que a lista deve ser tratada literalmente como dados, e não como uma instrução de código ou chamada de função para ser avaliada. Sem o apóstrofo, o Scheme tentou avaliar a lista como uma chamada de função, assumindo que A era o nome de uma função e que B e C eram os seus argumentos, gerando um erro.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
Expressão: (car (cdr (cdr '(A B C))))
Resultado: C
Explicação: O primeiro cdr descarta o A e retorna (B C). O segundo cdr descarta o B e retorna
(C). Por fim, o car extrai o elemento de dentro dessa lista restante, resultando em C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 3: comum em 6 passos, até 2 multiplicações esperando (expressão de até 24 caracteres); em cauda em 5 passos, nada esperando (até 16 caracteres).
n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).
n = 6: comum em 12 passos, até 5 multiplicações esperando (expressão de até 42 caracteres); em cauda em 8 passos, nada esperando (até 18 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

vai funcionar em ambos

Saída do Scheme

```
#t
#f
```

```
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
500
Traceback (most recent call last):
  File "/box/main.py", line 9, in <module>
    print(conta_ate(1_000_000, 0))
          ^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, o que cresce é a quantidade de operações pendentes na pilha de execução (e o tamanho das expressões intermediárias, como as multiplicações à espera). Isso acontece porque a operação principal (a multiplicação) fica guardada à espera que as chamadas recursivas mais profundas retornem um valor. Na recursão em cauda, como o resultado parcial é acumulado diretamente num parâmetro da função, a chamada recursiva é a última instrução executada, logo nada fica à espera e a pilha de chamadas não acumula novos registos

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme conseguiu realizar um milhão de chamadas recursivas porque as linguagens estritamente funcionais implementam a otimização de chamadas em cauda (Tail Call Optimization - TCO). Com esta otimização, o interpretador reutiliza o mesmo registo de ativação na pilha em vez de criar um novo para cada chamada, mantendo o consumo de memória constante. Por outro lado, o Python não faz esta otimização de forma nativa e possui um limite fixo de segurança para a profundidade da pilha (por defeito, 1000 chamadas), o que faz com que ultrapasse esse limite e lance um RecursionError.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (cond ((null? lis) 0)
        (else (+ 1 (tamanho (cdr lis))))))
```

```
(define (soma-quadrados lis)
  (cond ((null? lis) 0)
        (else (+ (* (car lis) (car lis)) (soma-quadrados (cdr lis))))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis)) (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais dificil foi a função pares, pois exige verificar uma condição adicional antes de decidir se junta (cons) o elemento atual ao resultado da recursão ou se o descarta.
Em todas as três funções, as primitivas foram aplicadas seguindo o padrão de recursão em listas do Scheme:
null?: Foi utilizado na condição base de cada função para verificar se a lista já terminou (se está vazia) e interromper a recursão.
car: Foi utilizado para aceder e processar o elemento atual da lista (obter o número para elevar ao quadrado em soma-quadrados ou para testar com even? e juntar com cons em pares).
cdr: Foi utilizado no passo recursivo de todas as funções para passar o resto da lista para a chamada seguinte.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares: Vai imprimir os primeiros 5 quadrados da lista infinita: [0, 1, 4, 9, 16].

Saídas (original e com member 17)

```
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42


com o print
Time limit exceeded
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell consegue processar listas infinitas devido à avaliação preguiçosa (lazy evaluation): os elementos só são calculados e gerados à medida que são estritamente necessários para o resultado final. Funções como take 5 pedem apenas os primeiros 5 elementos, fazendo com que o gerador pare imediatamente após obtê-los. primeiro 42 (div 1 0) não deu erro porque a função primeiro (primeiro x _ = x) descarta o segundo argumento, ignorando-o por completo. Como a expressão div 1 0 nunca precisa de ser avaliada, o erro de divisão por zero não chega a disparar.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

O que aconteceu: member 17 squares entra em loop infinito e nunca termina. O número 17 não é um quadrado perfeito, e como a lista de quadrados é infinita, a função continua a procurar indefinidamente para a frente. O member 16 terminou porque encontrou o 16 precocemente. Como corrigir: Como a lista de quadrados (squares) está estritamente ordenada de forma crescente, podemos parar a busca assim que o elemento atual da lista se tornar maior do que o número procurado, pois ele já não poderá aparecer mais à frente. Uma solução inspirada no livro (p. 670) verifica essa condição de parada antecipada:

```
member_ord b (a:x)
 | a == b    = True
 | a > b     = False
 | otherwise = member_ord b x
```

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5): Vai imprimir 15. A função somador(10) fixou n = 10, entao mais10(5) calcula 5 + 10.

Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

A função devolvida lembra-se do valor 10 devido ao mecanismo de fechamento (closure). Quando a função interna é criada dentro de somador, ela captura e guarda uma referência ao ambiente léxico onde foi definida, mantendo acesso às variáveis desse escopo (como o parâmetro n = 10) mesmo após a função externa somador ter terminado a sua execução

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

f.compose(g).apply(4) = $f(g(4))$: Aplica primeiro g e depois f ao resultado. $g(4) = 3 \times 4 = 12$, e depois $f(12) = 12 + 2 = 14$. f.andThen(g).apply(4) = $g(f(4))$: Aplica primeiro f e depois g ao resultado. $f(4) = 4 + 2 = 6$, e depois $g(6) = 3 \times 6 = 18$. Correspondência: O f.compose(g) é o que corresponde ao compor(f, g) da Prática 1 (onde $h(x) = f(g(x))$).

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
// Prática 7 - Exercício 2 reescrito com filter e map
const alunos = [
  { nome: "Ana", nota: 8.5 },
```

```
    { nome: "Bruno", nota: 5.0 },
    { nome: "Carla", nota: 7.0 },
    { nome: "Davi", nota: 9.5 },
  ];

  const aprovados = alunos
    .filter((aluno) ⇒ aluno.nota ⩾ 6)
    .map((aluno) ⇒ aluno.nome.toUpperCase());

  console.log(aprovados.join(", "));
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
# Prática 7 - Exercício 3 reescrito com compreensão de lista
precos = [50.0, 120.0, 80.0, 200.0]

com_desconto = [p * 0.9 if p > 100 else p for p in precos]

print(com_desconto)
```

Saídas das suas versões

```
ANA, CARLA, DAVI

[50.0, 108.0, 80.0, 180.0]
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

Aqui tens uma resposta mais direta e simplificada para a Questão 7.3:Questão 7.3Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?
Resposta:Mais fácil de ler: A versão funcional. Ela é mais limpa e declarativa, pois mostra diretamente o que está a ser feito em cada passo (filter -> map) sem precisar de controlar variáveis de laço.
Mais fácil de depurar: A versão com laço. É mais simples colocar um print ou um ponto de interrupção dentro do laço para ver o valor de uma variável em cada iteração do que inspecionar valores no meio de uma cadeia de funções.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

Versão funcional é mais clara. Aceitaria o custo em sistemas web/comerciais onde a clareza importa mais; não aceitaria em motores de jogos ou sistemas críticos de tempo real

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

A capacidade do Haskell processar listas infinitas sem esgotar a memória.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro era b <- h em vez de b <= h. O símbolo <- confundiu o compilador, fazendo-o inferir que o pivô era uma lista